

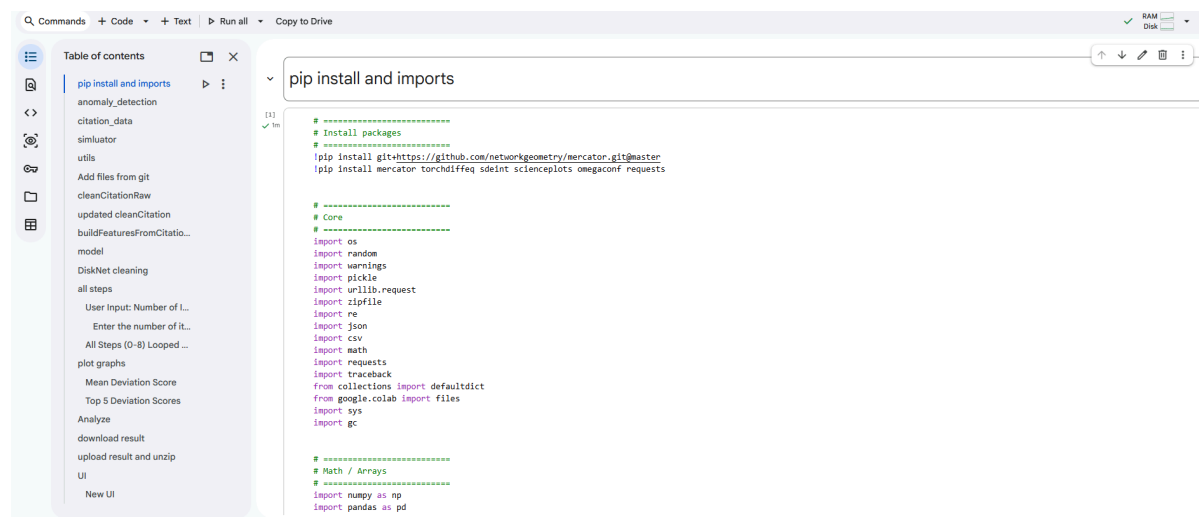
User Guide: DiskNet-Based Anomaly Detection System

Introduction

This document provides a comprehensive user guide for operating the DiskNet-based system designed for anomaly detection in dynamic citation networks. The system is executed via a Google Colab notebook and provides an interactive interface for data upload, processing, analysis, and visualization.

1. Opening and running the System Notebook:

- Open the provided `.ipynb` file using **Google Colab**.
- From the **Runtime** menu, select **"Run All"**.
- Wait until the execution completes.



```
# -----  
# Install packages  
# -----  
!pip install git+https://github.com/networkgeometry/mercator.git@master  
!pip install mercator torchdiffeq sdeint scienceplots omegaconf requests  
  
# -----  
# Core  
# -----  
import os  
import random  
import warnings  
import pickle  
import urllib.request  
import zipfile  
import re  
import json  
import csv  
import math  
import requests  
import traceback  
from collections import defaultdict  
from google.colab import files  
import sys  
import gc  
  
# -----  
# Math / Arrays  
# -----  
import numpy as np  
import pandas as pd
```

▶ Run all ▼

2. Accessing the User Interface (UI):

Once the execution is complete:

- Scroll to the bottom of the Colab notebook
- Locate the **User Interface (UI)** section.

...

Citation Data Portal

Go to File Upload

Check Results

3. Data Upload:

The system requires two input files: a metadata file describing the nodes and a citation edges file describing the relationships between them.

On the main screen, click "**Go to File Upload**" to begin the process.

4. **Uploading Citation Network Data:** Upload the required files according to the specifications:

Step 1: File Upload

1. Upload Metadata (CSV)

2. Upload Edges (CSV)

Check for Existing Files

Back to Home

4.1 Metadata File Requirements:

Upload a metadata file containing the following mandatory columns:

- **id**: Unique identifier for the paper.
- **year**: Publication year.
- **concepts**: Research concepts/tags.
- **primary_topic**: The main topic of the paper.

Important constraints:

- The publication years must be **continuous, with** no gaps.
- The dataset must span **exactly 21 consecutive years**.

Step 1: File Upload

1. Upload Metadata (CSV)

2. Upload Edges (CSV)

Check for Existing Files

[Back to Home](#)

Please select your meta file...

No file chosen

Step 1: File Upload

1. Upload Metadata (CSV)

2. Upload Edges (CSV)

Check for Existing Files

[Back to Home](#)

Please select your meta file...

No file chosen

Upload widget is only available for

Saving papers_metadata.csv to papers_metadata.csv

✔ Metadata Validated and Saved: Metadata Validated

4.2 Citation Edges File Requirements:

Upload an edges file describing citation relationships with the following columns:

- **cites**: The ID of the citing paper.
- **cited**: The ID of the paper being cited.

Step 1: File Upload

1. Upload Metadata (CSV)

2. Upload Edges (CSV)

Check for Existing Files

[Back to Home](#)

Please select your edges file...

No file chosen

Step 1: File Upload

1. Upload Metadata (CSV)

2. Upload Edges (CSV)

Check for Existing Files

5. Data Cleaning and Processing:

After uploading the files, the system automatically navigates to the data cleaning stage.

Step 2: Data Cleaning

[Back](#)

Click the "**Clean & Process Data**" button to initiate preprocessing.

Step 2: Data Cleaning

Clean & Process Data

Back

⌚ Starting full data cleaning process...
Removed self-citations: 224 rows dropped
After dropping duplicate edges: 24814 rows
Unique IDs appearing in edges: 6503
Removed isolated articles: 713 rows dropped
Filtered edges to valid meta IDs: 0 rows dropped
Corrected citation direction based on year: 22809 edges swapped
Final edges count: 24763
Saved cleaned edges to: /content/data/citation/clean_edges.csv
Saved cleaned meta to: /content/data/citation/clean_meta.csv
Saved /content/data/citation/id2idx_clean.json (N=6503)
✅ Data cleaning complete!

6. Model Execution:

- The system proceeds automatically to the model execution phase.
- Processing may take several minutes, depending on the dataset size
- Upon completion, the system redirects to the analysis screen.

Step 3: Ready for Analysis

Back

Start Analysis

Step 4: Pipeline Execution

Running 1 iterations...

▶ [Run 1/1] Initializing...

```
=====
Starting Run 1/1
=====
Data directory for this run: /content/data/citation_run_1
Log directory for this run: logs/citation_run_1/l3_h3/CitationNetwork/DiskNet

=====
STEP 0: Synthetic Citation Network Generator (Iteration-specific)
=====
Loading data files...
Loaded clean_meta.csv: 6503 rows
Loaded clean_edges.csv: 24763 rows
Loaded topics and concepts: 7 topics

Applying pre-cleaning logic to original graph data...
Initial clean graph: 6503 nodes, 24763 edges
After removing isolates: 6503 nodes
After removing self-loops: 24763 edges
After keeping LWCC: 6250 nodes, 24584 edges
Pre-cleaning complete. Final original graph size: 6250 nodes, 24584 edges

Calculating network parameters...

Creating synthetic nodes...
Created 623 synthetic nodes
```

7. Analysis and Visualization:

On the analysis screen, you can review the processed results and visualizations:

Step 5: Analysis & Results

Analyze SummaryGenerate Plots

Back to Home

7.1 Results Analysis:

Review anomaly scores and evaluation metrics produced by the model.

Step 5: Analysis & Results

Analyze SummaryGenerate Plots

Run	Total Nodes	Anomalies	Synth Recall %	Original Anom %	
0	1	6872	1341	74.317817	14.050248

Back to Home

7.2 Visualization:

Select your preferred graph visualization from the menu to explore the detected anomalies.

Step 5: Analysis & Results

Analyze SummaryGenerate Plots

✔ All plots regenerated with full labels and thresholds.

Select Plot Folder:

Back to Home

Analyze Summary

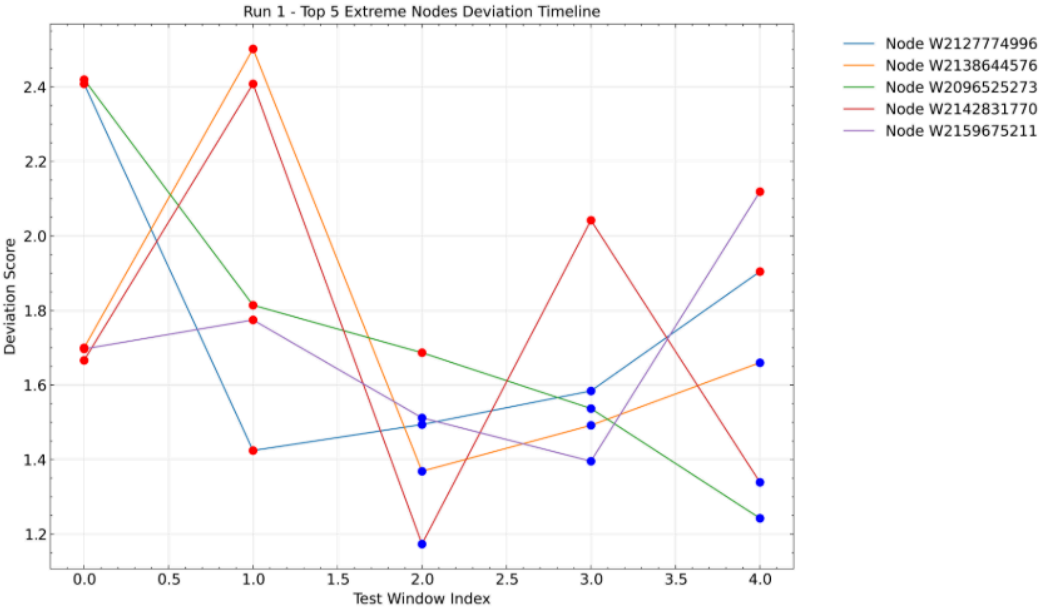
Generate Plots



Select Plot Folder:

Showing all plots from: **top_5_deviation_scores**

File: run_1_deviation_scores.png

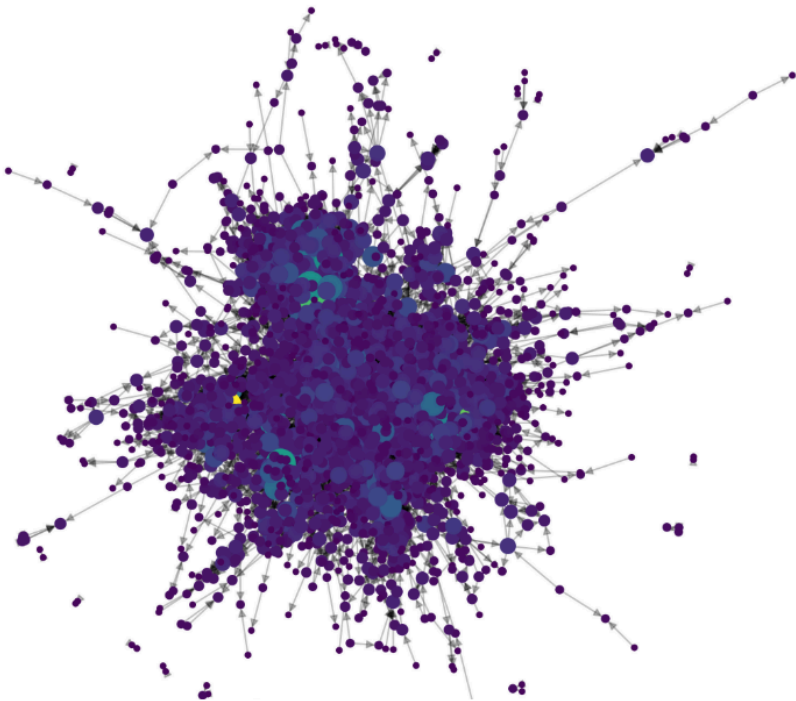


[Back to Home](#)

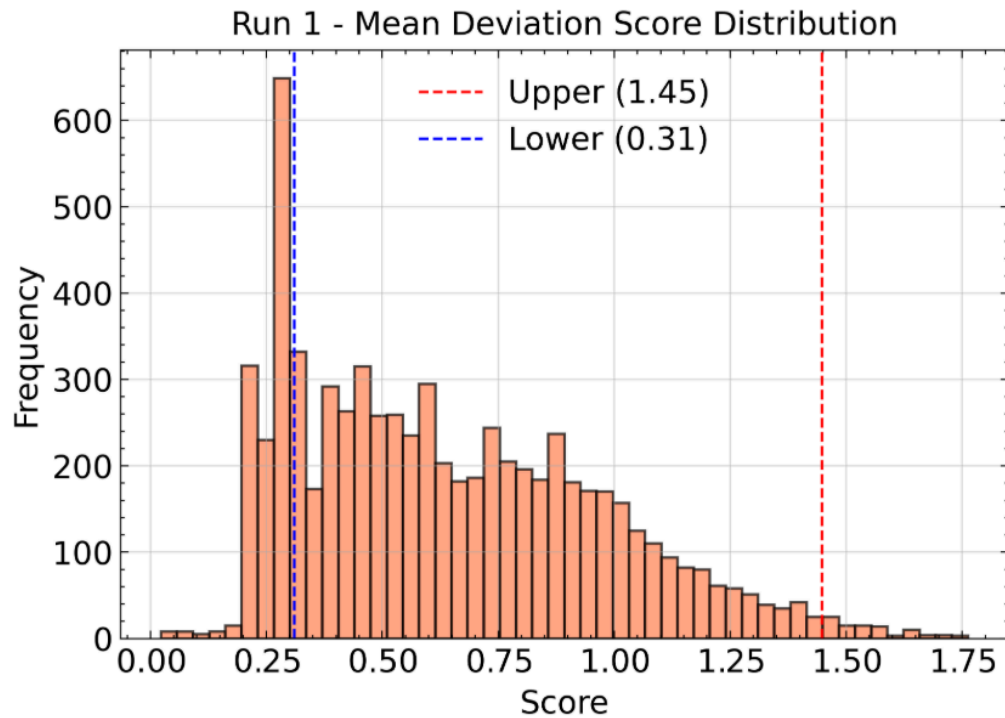
Showing all plots from: **graph**



File: run_1_graph.png



File: run_1_mean_scores_hist.png



Maintenance Guide: DiskNet for Citation Networks

1. System Environment

This section describes the necessary software infrastructure to run and maintain the project.

Software Requirements

- **Language:** Python 3.8 or higher

Core Libraries:

- **torch** (PyTorch) - For the DiskNet model and neural network operations.
- **networkx** - For graph manipulation and network analysis.
- **pandas & numpy** - For data processing and numerical calculations.
- **scikit-learn** - Specifically for the **IsolationForest** anomaly detection.
- **mercator** - External tool for hyperbolic embedding (installed via GitHub).

Installation Instructions

To set up the environment, run the following commands in your Python environment:

```
# Install core dependencies
pip install torch networkx pandas numpy scikit-learn matplotlib

# Install the Mercator library for hyperbolic mapping
pip install git+https://github.com/networkgeometry/mercator.git@master
```

2. Key Maintenance & Customization Points

For future research or adaptation to different datasets, the following modules and parameters are the most critical to modify:

A. Feature Engineering (`build_features_from_citation_csv`)

- The system's performance heavily relies on how node dynamics are represented.
- **Modifying Dynamics:** Currently, it extracts citation-based counts over time. It can be adapted to any time-series data associated with nodes and change the type of dynamics analyzed by modifying the logic within this function.
- **Quantity of Dynamics:** You can increase or decrease the number of features per node. If you change the input vector size, ensure the `input_dim` in the model configuration is updated accordingly to match the new feature set.

✓ buildFeaturesFromCitationCsv

[9]

```
def build_dynamics_from_graph(G, meta_csv, out_dir):
    os.makedirs(out_dir, exist_ok=True)

    # 1. LOAD METADATA
    meta_df = pd.read_csv(meta_csv, dtype=str)
    meta_df.columns = [c.strip() for c in meta_df.columns]
    meta_df["id"] = meta_df["id"].astype(str)

    # 2. LOAD GRAPH ID MAPPINGS
    graph_map_path = os.path.join(out_dir, 'graph_node_map_int_to_str.json')
    with open(graph_map_path, "r") as f:
        int_to_orig_id_map = {int(k): v for k, v in json.load(f).items()}
        orig_id_to_int_map = {v: k for k, v in int_to_orig_id_map.items()}

    # 3. PREPARE METADATA ARRAYS
    N = G.number_of_nodes()
    id2idx = orig_id_to_int_map
    years = np.full(N, -1, dtype=int)
    primary_topics = [None] * N

    for _, row in meta_df.iterrows():
        if row["id"] in id2idx:
            idx = id2idx[row["id"]]
            try:
                years[idx] = int(row["year"])
            except:
                years[idx] = -1
            primary_topics[idx] = row.get("primary_topic", None)

    # 4. PRE-CALCULATE GRAPH DISTANCES (for f4)
    # We use an undirected version of the graph to find 'community distance'
    G_undirected = G.to_undirected()
```

B. Global Configuration (Config class)

The `Config` class centralizes all hyperparameters for the time-series forecasting. Adjust these to tune the model's "vision":

- **Lookback:** Determines how many past time steps the model considers for prediction.
- **Horizon:** Determines how many steps into the future the model attempts to predict.
- **Data Ratios:** The `train_ratio`, `val_ratio`, and `test_ratio` control the data split. Adjust these if the dataset size changes significantly to ensure the model has enough samples to learn effectively.

```
# Fixed Parameters
L_FIXED = 3
H_FIXED = 3
K_FIXED = 1
TRAIN_RATIO_FIXED = 0.6
VAL_RATIO_FIXED = 0.15
```

C. Synthetic Anomaly Generation

The system evaluates its detection capabilities by injecting a synthetic network into the original data.

- **Network Size:** Controlled by the ratio of synthetic nodes (default: 10% of the total network). Increasing this will make the anomaly detection task more prominent but potentially less realistic.
- **Connectivity:** Density relative to the original graph (default: 30% of the original network's density). This ratio dictates how "hidden" or "obvious" the synthetic cluster is within the global graph structure. Adjusting this parameter allows you to test the model's sensitivity to different structural anomalies.

```
def calculate_network_parameters(clean_meta_df: pd.DataFrame, clean_edges_df: pd.DataFrame):  
    print("\nCalculating network parameters...")  
    original_nodes = len(clean_meta_df)  
    new_nodes_total = math.floor(original_nodes * 0.1)  
    original_edges = len(clean_edges_df)  
    new_edges_total = math.floor(original_edges * 0.3)  
    return new_nodes_total, new_edges_total
```

3. Maintenance Procedures

1. **Retraining:** Full retraining of the DiskNet model is required after any significant change to `Lookback`, feature definitions, or `input_dim` (input dimensions).
2. **Threshold Tuning:** The `contamination` parameter in the `CitationAnomalyDetector` should be adjusted based on the expected anomaly ratio in the specific real-world use case.